

PRISM: Predictive Runtime In-place Scaling and Model Selection for Edge Microservices

Uwe Gropengießer^{*} , Thomas Reuter^{*}, Dominik Schön^{*} , Osama Abboud[†] , Xun Xiao[†]
Max Mühlhäuser^{*} 

^{*} Technical University of Darmstadt; Darmstadt, Germany

[†] Huawei Heisenberg Research Center Munich; Munich, Germany

Abstract—Latency-sensitive edge AI services must balance strict deadlines, output quality, and limited compute and energy budgets. However, static CPU provisioning wastes resources because inference cost varies substantially across inputs, model variants, and runtime conditions. We present PRISM, a prediction-guided runtime framework that jointly selects model variants and CPU allocations for containerized edge microservices. Using container-level energy monitoring and lightweight regression models, PRISM adapts each pipeline stage in place and minimizes predicted CPU-package energy under deadline, resource, and offline model-level Quality of Result (QoR) constraints. We evaluate PRISM on more than 52,000 requests in an Automatic License Plate Recognition (ALPR) pipeline with detection and recognition stages. For detection, PRISM reduces energy consumption by 36 % compared to the strongest static configuration while preserving a comparable success rate and using less than half of the average CPU allocation. For recognition, it reaches near-static-best performance with lower average CPU allocation. These results show that predictive in-place adaptation is a practical mechanism for making time-sensitive AI microservice pipelines more energy-efficient at the edge.

Index Terms—Energy-aware Vertical Scaling, Edge Computing, Approximate Computing, Quality of Result

I. INTRODUCTION

Edge computing relocates computation and storage closer to data sources, reducing latency, lowering bandwidth usage, and enabling real-time analytics [1], [2]. At the same time, AI workloads at the edge must operate under tight compute and energy budgets on resource-constrained devices [3]. Many edge inference workloads are latency-sensitive, with Automatic License Plate Recognition (ALPR) in traffic enforcement and access control as a prominent example where late or inaccurate results can hinder incident response or disrupt operations. Consequently, such workloads must respect deadlines, limit energy consumption, and meet application-specific Quality of Result (QoR) requirements, which capture task-specific output quality. These requirements differ across domains. Some applications demand near-perfect precision, while others can tolerate approximate results. This trade-off has been widely studied in the context of approximate computing [4], [5], [6]. In distributed and resource-constrained edge deployments, balancing execution time, energy consumption, and output quality remains difficult because inputs, resource availability, and runtime conditions vary substantially. Our work addresses this setting by making energy–time–quality

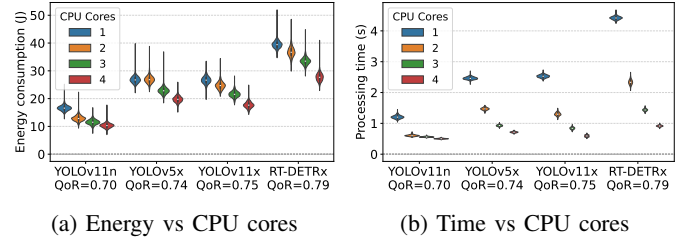


Fig. 1: Vertical scaling via CPU-core allocations. Distributions of (a) energy and (b) execution time for RT-DETRx, YOLOv11n, YOLOv11x, and YOLOv5x, based on 15,118 executions at 6960×4640 pixels (32 MP). The measurement methodology follows prior measurement work [7].

trade-offs explicit and actionable for containerized edge AI services.

A first illustration of this trade-off is given in Figure 1, which shows the impact of vertical CPU scaling [8] on the energy consumption and execution time of different detector variants. The violin plots highlight how model choice and CPU allocation interact. Larger variants such as YOLOv11x, YOLOv5x, and RT-DETRx consume more energy and run longer than smaller variants such as YOLOv11n, even under the same CPU allocation. This reflects the cost of using more accurate models. Vertical CPU scaling, in contrast, primarily reduces execution time within a fixed model. As shown in Figure 1b, additional CPU resources shift execution-time distributions toward shorter runtimes. The corresponding energy distributions in Figure 1a change less uniformly, indicating that faster execution does not automatically translate into proportional energy savings, consistent with prior observations on power–performance trade-offs under vertical scaling [9]. Scaling can therefore reduce latency, while model selection changes both quality and resource demand. Energy-efficient edge inference must consequently consider model selection and CPU allocation jointly, while enforcing explicit deadline and quality constraints.

Prior research has explored both approximation and scaling as mechanisms for resource optimization. MobiDiC [10] exploits approximation to trade result quality for reduced execution time in mobile distributed computing, while MobiQoR [11] introduces QoR as an optimization dimension for task offloading decisions between local and remote execution.

Further work demonstrates the benefits of vertical scaling for meeting Quality of Service (QoS) objectives in AI-based applications [12], [13], [14], and the energy footprint of AI models has attracted growing attention [15], [16], [17]. Most closely related, MARQ [18] enables dynamic adaptation of AI service chains at runtime by selecting among alternative execution paths to jointly optimize QoR, execution time, and energy. However, MARQ and related approaches optimize at the service-chain level and treat the allocation of each microservice stage as fixed during execution. They therefore leave open how a stage should choose among model variants and CPU allocations per request, how these choices jointly affect latency and energy, and how this relationship can be learned from lightweight pre-execution features and exploited at runtime. The combination of prediction-guided model selection and vertical CPU scaling under explicit QoR and deadline constraints remains insufficiently explored for containerized edge microservices. At the same time, container-level energy monitoring through Kepler and RAPL makes the energy effects of such decisions observable at runtime [19], [20].

In this paper, we present PRISM, a prediction-driven runtime framework for energy-aware adaptation of containerized edge microservices. Building on prior measurement evidence that per-request execution time and energy under vertical CPU scaling are predictable [7], PRISM closes the loop from measurement and prediction to runtime enforcement. It combines regression-based performance modeling, in-place vertical CPU scaling, and per-stage model selection in Kubernetes. A key novelty is the explicit treatment of QoR as a stage-local runtime constraint rather than optimizing latency or throughput alone. For each request and microservice stage, PRISM selects the configuration that minimizes predicted CPU-package energy while satisfying a deadline, a resource bound, and an offline model-level QoR threshold. These offline quality scores avoid the need for ground-truth labels during operation, while predictive models trained from container-level measurements estimate execution time and energy before execution. The selected configuration is then applied in place without restarting the pod.

We evaluate PRISM in a realistic ALPR pipeline comprising object detection and Optical Character Recognition (OCR) stages. Our experiments show that prediction-guided, stage-local adaptation can substantially reduce energy consumption while preserving success rates comparable to the strongest static configurations. The results demonstrate that combining predictive modeling with in-place scaling and model selection provides a practical mechanism for energy-efficient execution of time-sensitive AI microservice pipelines at the edge.

After introducing the motivating example and background in Section II, the remainder of this paper is structured as follows.

- Section III presents the closed-loop system design, including container-level measurement, offline model training, runtime monitoring, and in-place adaptation of microservice stages.

- Section IV formalizes the prediction and selection method, including feature construction, predictor deployment, and per-stage constrained selection of model variant and CPU allocation.
- Section V presents the dataset and research-question-driven evaluation, covering the effects of vertical scaling, prediction accuracy, operational feasibility, and end-to-end behavior on the ALPR pipeline.

Finally, we discuss insights and limitations in Section VI, review related work in Section VII, and conclude in Section VIII.

II. BACKGROUND AND MOTIVATING EXAMPLE

We use ALPR as a motivating example to illustrate the challenges of joint model selection and resource allocation in edge-based AI applications. Figure 2 shows an ALPR pipeline deployed at the edge. We use *stage* to refer to a logical step in the pipeline, such as detection or OCR, and *microservice* to describe its containerized implementation. The pipeline consists of three stages implemented as microservices, namely object detection, OCR, and a database for license plate validation. We refer to the first two stages as DETECT and OCR. DETECT and OCR offer multiple model variants. For example, DETECT may use YOLOv11n or RT-DETRx, while OCR can rely on lightweight or more advanced recognizers. These choices affect execution time, energy consumption, and QoR. Smaller models usually reduce latency and energy consumption, whereas larger variants can improve output quality at a higher computational cost.

PRISM treats each stage as an independent adaptation unit. For every request and stage, it selects a model variant and CPU allocation that minimizes predicted CPU-package energy under that stage’s deadline, resource bound, and offline model-level QoR threshold. This stage-local design keeps the runtime decision bounded and predictable and complements application-level orchestration frameworks such as MARQ [18], which operate at the chain level but assume fixed resource provisioning per stage.

While smaller models generally achieve lower execution time and energy consumption, they may fail to provide sufficient quality in accuracy-sensitive applications. In such cases, higher QoR can justify increased energy use, particularly when latency budgets can still be met through resource scaling. This trade-off underscores the need to jointly consider model selection and resource allocation rather than treating them in isolation. This raises the central question of which variant to select and how to adjust resource allocation to balance output quality, latency, and energy consumption under given constraints.

The runtime selector depends on model-level QoR constraints and per-request measurements of execution time and CPU-package energy. The following subsections define these signals and explain how they are obtained.

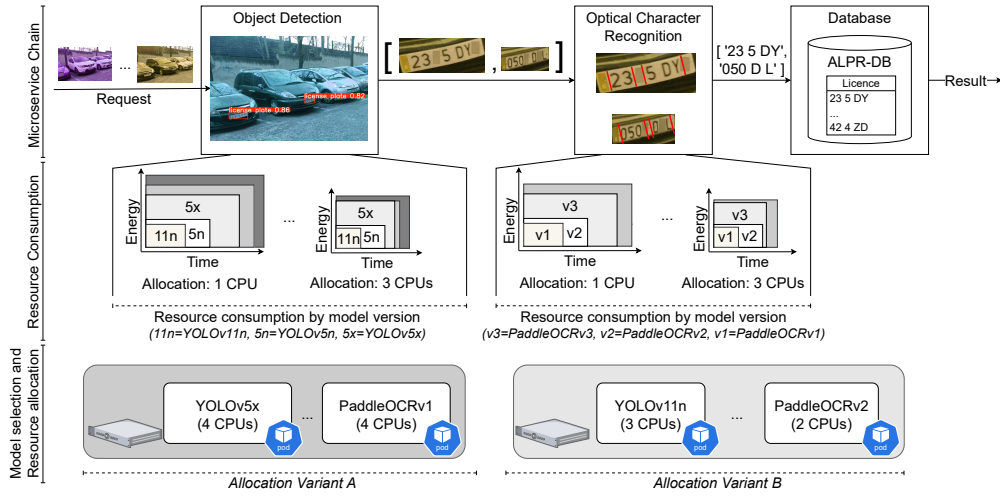


Fig. 2: Exemplary presentation of the “Automated License Plate Recognition” pipeline. It consists of three stages implemented as microservices. Object detection and OCR support multiple model variants and CPU allocations, while the database stage remains fixed. Different stage assignments and model choices lead to varying energy consumption and execution times.

A. Quality of Result

QoR is a normalized, percentage-based measure that quantifies the expected output quality of a computation step relative to a task-specific reference metric [10], [18]. Its key advantage is that it provides a uniform abstraction for expressing quality requirements across application domains. In computer vision, common metrics include Mean Average Precision (mAP) for object detection [21] and word-level accuracy for text recognition [22]. In data analytics, relative error is often used to characterize the quality of approximation [23], [24], [25].

The QoR of a service can in principle be determined either offline or online, depending on the availability of ground truth. In this paper, offline refers to evaluation on annotated datasets during training and benchmarking. Online refers to runtime operation on unlabeled live inputs. Offline assessment yields precise and reproducible quality measurements, but is limited to controlled datasets. During online operation, ground truth is typically unavailable. Runtime QoR can therefore only be approximated indirectly through proxy signals that correlate with accuracy, such as prediction confidence or temporal consistency in video streams. Although such signals are useful in practice, we do not use them here because their calibration without ground truth is task-specific, deployment-dependent, and difficult to validate across systems.

Instead, we derive per-model offline QoR scores using mAP for DETECT and word-level accuracy for OCR. At runtime, these scores are used as minimum model-level quality constraints during configuration selection. This design enables reproducible and deployment-independent decisions without relying on task-specific confidence calibration or online accuracy estimation. It does not provide a per-request quality guarantee. Rather, it ensures that selected configurations use models whose offline reference quality satisfies the requested threshold.

B. Container-Level Energy Monitoring in Kubernetes

Precise measurement of execution time and energy is crucial for energy–time–quality-aware adaptation. In this work, vertical scaling refers to adjusting the CPU resources available to a pod and does not cover GPU or TPU resources. We instrument our Kubernetes cluster with Kepler [19], which attributes Intel Running Average Power Limit (RAPL) [20] CPU-package energy counters to individual containers. Figure 3 illustrates the node-level measurement and attribution path used at runtime. Kepler ingests RAPL package energy and per-pod CPU usage, and attributes package-level energy to pods based on cgroup statistics. PRISM samples these attributed counters immediately before and after each request to derive per-request execution time and CPU-package energy.

Kepler correlates package-level energy from RAPL domains with per-pod CPU usage obtained from Linux cgroups, deriving container-level energy estimates without additional hardware instrumentation. For example, if a pod accounts for 60% of CPU usage in an interval where the CPU package consumed 10J, Kepler attributes 6J to that pod. This attribution mechanism provides the per-request energy signal on which PRISM’s runtime decisions are based.

We report CPU-package energy because CPU allocation is the control knob used by PRISM and directly affects execution time, deadline feasibility, and the CPU energy consumed during inference. This focus is also consistent with prior large-scale microservice analysis, which shows that microservice runtime performance is substantially more sensitive to CPU interference than to memory interference [26]. CPU-package energy, therefore, captures the decision-dependent component most directly affected by PRISM’s in-place scaling decisions. Full-system energy additionally includes components such as DRAM, storage, networking, and static platform baselines. These components remain relevant for full-system accounting,

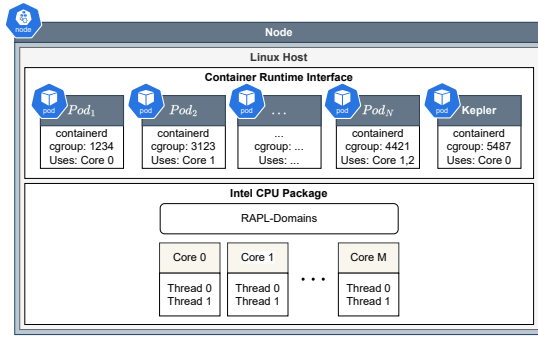


Fig. 3: Node-level measurement and attribution path for container energy. Kepler maps CPU-package energy from RAPL counters to pods using per-pod CPU usage, enabling per-request energy accounting in PRISM.

but they are not directly controlled by the CPU allocation decisions studied here. Prior work has demonstrated that RAPL-based CPU energy measurements provide a practical foundation for energy-aware algorithm engineering, while non-CPU domains and container-level power models require platform-specific support or modeling and can be less comparable across systems [27], [28].

Our evaluation, therefore, focuses on CPU-based in-place vertical scaling. In-place scaling of accelerator resources is outside the scope of this work. Kubernetes In-Place Pod Resize currently targets CPU and memory resources, whereas accelerators such as GPUs or TPUs are managed through device-specific mechanisms and do not expose the same generic in-place resizing interface [29].

C. Model Preloading and In-Place Adaptation

PRISM assumes that all candidate model variants of a microservice stage are preloaded and kept in memory during operation. This design enables in-place adaptation because model switching does not require loading a new model from storage and can be performed on a per-request basis. Keeping models in memory reduces switching latency and avoids model-loading cold starts during request processing, which is important for deadline-sensitive inference.

The static memory footprint of preloaded model weights is not part of PRISM’s optimization objective. Instead, PRISM optimizes per-request, decision-dependent costs within this operating mode, namely execution time and CPU-package energy. Activation tensors and temporary workspace memory are allocated per inference and released afterwards, while model parameters contribute to the persistent footprint. This assumption is practical when the candidate set fits into memory, but it can limit applicability on highly memory-constrained edge devices. Managing model loading, eviction, placement, or container lifecycle decisions is orthogonal to PRISM’s scope and can be handled by the underlying orchestration layer.

III. SYSTEM DESIGN

To enable dynamic, prediction-driven adaptation of microservices at runtime based on execution time, energy con-

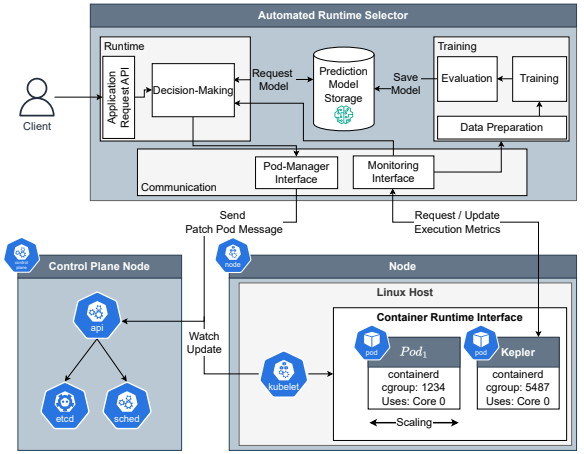


Fig. 4: System design of PRISM. The measurement subsystem captures per-request execution time and CPU-package energy via Kepler. Offline preprocessing transforms traces into regression models. At runtime, PRISM filters candidate configurations by offline QoR and deadline constraints, then applies the energy-minimal selection via in-place CPU scaling and model switching.

sumption, and quality constraints, we design an integrated control loop that adapts containerized edge applications in place. Figure 4 illustrates the overall architecture. The right-hand side depicts the measurement subsystem, which captures fine-grained execution time and energy metrics at the container level on edge hardware. The left-hand side shows the runtime adaptation loop, which evaluates feasible configurations per request and applies the selected one through in-place reconfiguration. Between these two components, an offline preprocessing pipeline transforms collected measurements into predictive models that form the basis for online decision-making. Together, these elements implement control of microservices under realistic operating conditions with awareness of energy, time, and quality. The design deliberately focuses on decision-dependent, per-request costs that can be actuated at runtime. It therefore excludes global pipeline scheduling, model lifecycle management, and system-wide energy components that are not directly controlled by CPU scaling.

The measurement subsystem provides the data foundation for both offline model training and online decision-making. It is deployed as a Kepler DaemonSet on every Kubernetes node and leverages Intel RAPL counters to capture CPU package-level energy consumption. These readings are correlated with Linux cgroup statistics and CPU-affinity information obtained via the Container Runtime Interface (CRI), yielding per-pod time series of execution time and attributed CPU-package energy. While Kepler exposes metrics via Prometheus, the default scrape interval is too coarse for short-lived inference workloads. Therefore, PRISM queries Kepler’s metrics directly in Prometheus-exporter format and computes energy differences between consecutive readings at the smallest available interval. This per-request differencing captures millisecond-

to-second-scale workloads and, in combination with repeated executions, reduces the impact of transient system noise and outliers. Because energy attribution is derived from CPU-package counters and normalized by per-pod CPU usage, the resulting measurements are aligned with the vertical CPU scaling decisions made by PRISM. This makes the signal more suitable for runtime selection than system-level energy metrics whose static components would reduce sensitivity to CPU allocation changes.

Collected measurements are processed offline in a pre-processing pipeline. Traces are transformed into feature vectors that include the selected model variant, CPU allocation, and input characteristics such as image resolution and file size. Based on these features, regression models predict execution time and CPU-package energy. Result quality is incorporated via precomputed per-model QoR scores rather than inferred online. This enables reproducible decision-making without task-specific confidence calibration or ground-truth labels at runtime, but constrains quality at the model level rather than guaranteeing per-request quality. Candidate predictors include linear and polynomial regression, Multilayer Perceptron (MLP), XGBoost, and Bayesian regression. Model selection is primarily guided by Mean Absolute Percentage Error (MAPE), which reflects relative prediction error and is therefore relevant for per-request decisions. We additionally report R^2 , Mean Absolute Error (MAE), and Root Mean Squared Error (RMSE) for completeness. The selected predictors are serialized and stored in a model database together with metadata such as training time and observed accuracy. During operation, predicted and measured values are logged to support later offline retraining in the presence of workload drift.

At runtime, PRISM performs stage-local decision-making for each incoming request. For a given request and stage, all combinations of model variant m and CPU allocation c are enumerated. Candidate configurations are first constrained by offline model-level quality, then pruned by deadline feasibility, and finally ranked by predicted energy consumption. This ordering discards infeasible options early and focuses optimization on decision-dependent costs. For each candidate, the trained regression models predict execution time $\hat{T}(m, c)$ and energy consumption $\hat{E}(m, c)$, while quality is checked against the offline per-model score QoR_m . Configurations whose predicted execution time exceeds the request deadline $T^{\max}$ or whose offline quality score falls below the minimum threshold $\text{QoR}^{\min}$ are discarded. Among the remaining feasible options, PRISM selects the configuration with the minimum predicted energy. Decisions are made independently per microservice and per request, without redistributing deadlines or quality budgets across pipeline stages.

The selected configuration is applied *in place* using Kubernetes’ In-Place Vertical Scaling mechanism. The kubelet adjusts CPU allocations directly in the container’s cgroups without restarting the pod, preserving execution state. Model switching is likewise handled inside the running microservice. Each microservice preloads all supported model weights at initialization, and the active model is selected via an internal

switch at request time. After initialization, this design avoids model-loading delays during request processing. The per-request decision overhead is bounded by the deployed model variants, CPU options, and configured solver termination criteria. In our evaluation, even the more expensive XGBoost-Timefold configuration remains below the corresponding inference latency, as shown in Section V. By relying on CRI calls and in-place reconfiguration, PRISM supports adaptation without pod restarts and keeps runtime behavior consistent across requests.

After execution, the observed execution time and energy consumption are fed back into the measurement infrastructure, closing the control loop. Measurement, offline modeling, runtime prediction, and in-place actuation thus form an integrated system for adapting microservices to varying inputs and constraints on edge hardware. Because the same decision and actuation mechanism is applied independently at each microservice stage, it can be composed along multi-stage pipelines while preserving stage-local decision boundaries.

IV. APPROACH

Building on the system design in Section III, we formalize the predictive modeling and per-stage constrained selection that enable PRISM’s runtime adaptation. The approach addresses three interdependent concerns. First, it defines how pre-execution information is represented as features that capture the joint effect of model variant, CPU allocation, and input properties. Second, it defines how predictors for execution time and energy are trained and deployed. Third, it defines how the per-stage selection problem is solved at request time under deadline, offline quality, and resource constraints.

To predict runtime behavior, we map pre-execution information to features. For each request, we construct a feature vector for each candidate configuration by combining the CPU allocation c , the model variant m , and observable input properties such as image height h , width w , and file size b . These variables comprise the decision parameters (m, c) and the observed input properties (h, w, b) that influence performance. Together, they define the feature vector

$$\mathbf{z}(m, c, h, w, b) = [m, c, h, w, b].$$

QoR is deliberately not part of the feature vector because request-level ground truth is unavailable during live operation, as discussed in Section II. Estimating per-request QoR would require access to ground truth at runtime, which is not feasible in the target edge setting. All features used by PRISM are available before stage execution and can be extracted without running the selected model. This keeps the selector independent of model internals and allows decisions to be made before inference starts.

Instead, we precompute average quality scores QoR_m for each model on a labeled reference dataset and compare them against per-stage thresholds $\text{QoR}_k^{\min}$ provided by the application or request policy at runtime. This design supports reproducible selection decisions and avoids task-specific online proxies. It constrains the selected model by its offline reference

quality rather than guaranteeing the quality of every individual request.

Because the features differ in type and scale, preprocessing is required before training and inference. The categorical model identifier m is converted with one-hot encoding so that each architecture is represented by its own binary dimension, allowing downstream regressors to consume m as numeric input. The numeric variables (c, h, w, b) are standardized to reduce scale effects and improve numerical stability. The standardization parameters are fitted on the training data and reused unchanged at runtime to keep the training and inference distributions consistent.

Polynomial terms are only used when training explicit polynomial regression baselines. They provide linear models with additional flexibility to capture simple nonlinear relationships such as diminishing returns from additional CPU allocation. For a scalar input u , we use a polynomial feature map of degree d

$$\phi_d(u) = (u^1, \dots, u^d),$$

which adds powers of u only and does not include interaction terms. Using this notation, the full preprocessing map for the polynomial baselines is

$$\varphi(\mathbf{z}) = [\phi_d(c), \phi_d(h), \phi_d(w), \phi_d(b), \text{OHE}(m)].$$

No constant term is included in $\phi_d(\cdot)$ because the intercept is handled by the model’s bias parameter. For predictors such as XGBoost and MLPs, we apply the same preprocessing steps except that the polynomial terms are omitted, since these models capture nonlinear relationships through trees or learned hidden layers. For example, with polynomial degree $d = 3$ and CPU allocation $c = 2$, the expansion yields $\phi_3(c) = (2, 4, 8)$.

We evaluate a family of regression candidates reflecting different bias–variance trade-offs, including linear models, polynomial regression, Bayesian regression, MLPs, and XGBoost. XGBoost consistently yields the most accurate predictions for both execution time and energy in our experiments and is therefore deployed as the default predictor in PRISM. The other regressors are used as baselines in the evaluation and are not queried by the default runtime configuration. The full comparison is reported in Section V.

Beyond point accuracy, uncertainty can matter when enforcing strict deadlines. Bayesian regression provides credible intervals that quantify posterior uncertainty around a prediction. For $y \in \{E, T\}$ we assume

$$\begin{aligned} y \mid X &\sim \mathcal{N}(X\beta + \alpha, \sigma^2), \\ \alpha, \beta_k &\sim \mathcal{N}(0, \tau^2), \\ \sigma &\sim \text{HalfNormal}(\gamma), \end{aligned}$$

which yields point predictions $\hat{E}$ and $\hat{T}$ together with credible-interval half-widths ΔT . At runtime, deadlines can then be treated conservatively by requiring

$$\hat{T} + \Delta T \leq T_{\max}.$$

When a deterministic predictor such as XGBoost is deployed, there is no posterior by construction. In this case, ΔT can be

replaced by an empirical slack $\delta > 0$ derived from validation residuals, for example, the 95th percentile of $|T - \hat{T}|$ per configuration.

For each stage k , we denote by Ψ_k the deployed predictor that maps the preprocessed input $\tilde{\mathbf{z}}$ to estimates of energy, time, and an optional margin

$$\Psi_k(\tilde{\mathbf{z}}) = (\hat{E}_k, \hat{T}_k, \Delta T_k).$$

The margin ΔT_k represents either the half-width of a Bayesian credible interval or an empirical slack derived from validation residuals. In our evaluation, we set $\Delta T_k = 0$ and focus on point predictions. The interface nevertheless supports conservative deadline checks when a margin is enabled.

Training and model selection follow the needs of the runtime decision. All candidate models are tuned using k -fold cross-validation, with hyperparameters such as tree depth, learning rate, polynomial degree, and regularization strength varied accordingly. We use MAPE as the primary selection metric because it is scale-free across the DETECT and OCR targets. After selection, the final model for each target is retrained on the available data and exported together with all preprocessing metadata. Using the same artifacts at runtime ensures that inference applies the same preprocessing pipeline as training.

A. Pipeline Input Propagation

In a multi-stage pipeline, the input to a downstream stage depends on the output of an upstream stage. In the ALPR pipeline, the OCR stage receives bounding-box crops produced by DETECT rather than the original image. The size of these crops varies with scene content and cannot be known before DETECT has executed.

PRISM addresses this dependency through bucketed size estimation. Before DETECT executes, the OCR decision engine can map the original image resolution to a discrete bucket calibrated on training data. Each bucket stores representative crop dimensions observed during profiling. Once DETECT produces the actual crop, the measured crop dimensions replace the bucket estimate for OCR and any remaining downstream stages. This two-phase approach provides a fallback estimate before upstream outputs are available while using actual dimensions whenever they have been produced.

Bucketing introduces estimation error because the downstream input is only approximated before the upstream stage has completed. This error can be covered by the optional margin ΔT_k when conservative deadline enforcement is enabled. In our evaluation, we report the point-prediction setting with $\Delta T_k = 0$. Because OCR crops are typically much smaller than the original image, downstream prediction is less dominated by full-resolution image size than the DETECT stage.

B. Decision Overhead

Requests traverse a microservice chain of K stages. At each stage k , the decision engine considers all combinations of model variant $m \in \mathcal{M}_k$ and CPU allocation $c \in \mathcal{C}_k$. The remaining CPU budget C_{rem} is treated as an external bound

Algorithm 1: Sequential decision in a microservice chain with offline QoR and optional time margin ΔT_k .

Input: Input image I , stages L , per-stage candidates (M_k, C_k) , bounds $(T_k^{\max}, \text{QoR}_k^{\min})$, remaining CPU budget C_{rem}

Output: $(m_k, c_k)_k$ or FALLBACK

```

1 Extract  $h, w, b$  from  $I$ 
2 for  $k \in L$  in order do
3    $\text{candidates} \leftarrow []$ 
4    $p_k \leftarrow \text{stage\_inputs}(k, h, w, b)$ 
    // source-agnostic: raw tuple or derived
5   for  $m \in M_k$  do
6     if  $\text{QoR}_m < \text{QoR}_k^{\min}$  then
7       continue
8     for  $c \in C_k$  do
9       if  $c > C_{\text{rem}}$  then
10        continue
11       $\tilde{z} \leftarrow \varphi([c, p_k, m])$  // OHE( $m$ ) +
        standardization;  $\phi_d$  only for
        polynomial baselines
12       $(\hat{E}, \hat{T}, \Delta T_k) \leftarrow \Psi_k(\tilde{z})$  //  $\Delta T_k$ : Bayesian
        half-width or empirical slack
13      if  $\hat{T} + \Delta T_k > T_k^{\max}$  then
14        continue
15      append  $(m, c, \hat{E})$  to  $\text{candidates}$ 
16    end for
17  end for
18  if  $\text{candidates} \neq \emptyset$  then
19     $(m_k, c_k) \leftarrow \arg \min_{(m, c, \hat{E}) \in \text{candidates}} \hat{E}$ 
20    apply_config( $k, (m_k, c_k)$ )
21    execute_stage( $k$ )
22  else
23    return FALLBACK
24 end for
25 return  $\{(m_k, c_k)\}_{k \in L}$ 

```

supplied by the orchestration layer. PRISM does not decide cluster-wide placement, but uses this bound to avoid selecting configurations that exceed the resources made available to the stage. Given a per-stage deadline $T_k^{\max}$ and a minimum offline quality threshold $\text{QoR}_k^{\min}$, the engine solves

$$\begin{aligned}
& \min_{m \in \mathcal{M}_k, c \in \mathcal{C}_k} \hat{E}_k(m, c) \\
& \text{s.t. } \hat{T}_k(m, c) + \Delta T_k(m, c) \leq T_k^{\max}, \\
& \quad \text{QoR}_m \geq \text{QoR}_k^{\min}, \\
& \quad c \leq C_{\text{rem}}.
\end{aligned}$$

The objective minimizes predicted CPU-package energy among all feasible candidates. Execution time appears as a hard deadline, QoR as an offline model-level quality constraint, and the CPU budget as a strict resource bound. In the ALPR pipeline evaluated in Section V, the candidate space amounts to $|\mathcal{M}_k| \times |\mathcal{C}_k|$ configurations per stage, with $4 \times 5 = 20$ candidates for DETECT and $4 \times 8 = 32$ for OCR.

Algorithm 1 instantiates this selection as a conceptual enumeration over feasible candidates. In the current deployment, it is realized via Timefold [30] with a Hill Climbing local search that terminates early once no further improvement is found

within a configurable step limit. The empirical overhead of this implementation is characterized in Section V. Because the candidate set is bounded by the deployed model variants and CPU options, direct enumeration is equivalent and preferred for latency-critical deployments.

The loop proceeds stage by stage. It first extracts (h, w, b) from the incoming request and derives the stage-specific inputs p_k , either from the raw image tuple or from bucketed downstream sizes when k depends on an upstream output. For each model $m \in \mathcal{M}_k$ satisfying $\text{QoR}_m \geq \text{QoR}_k^{\min}$ and each CPU allocation $c \in \mathcal{C}_k$ with $c \leq C_{\text{rem}}$, the loop builds the preprocessed vector $\tilde{z} = \varphi([c, p_k, m])$ and queries the deployed predictor via Ψ_k to obtain $(\hat{E}_k, \hat{T}_k, \Delta T_k)$. Candidates with $\hat{T}_k + \Delta T_k > T_k^{\max}$ or violations of the budget or offline quality threshold are discarded. The remaining set is ranked by $\hat{E}_k$, and the minimizer (m_k, c_k) is selected.

The chosen configuration is enforced in place. The kubelet updates cgroup CPU limits without restarting the pod, and the active model is switched in process because all stage-specific weights are preloaded. After executing stage k , the observed outputs are propagated to the next stage and replace the bucketed estimates where applicable. C_{rem} is provided by the orchestrator and acts as a per-stage resource bound in the selection loop. If no feasible candidate exists, the request is not forwarded to the current stage and is recorded as a failed execution. This case arises when the specified deadline and offline QoR threshold cannot be satisfied by any available configuration, and is accounted for in the success rate reported in Section V.

Finally, measured energy and time are fed back into the monitoring subsystem and compared to the predictions. Deviations exceeding a tolerance threshold can trigger corrective actions such as offline retraining to maintain prediction accuracy under workload drift. This closes the loop of measurement, prediction, optimization, and enforcement.

V. EVALUATION

Edge AI workloads must simultaneously satisfy deadlines, energy budgets, and quality constraints under varying inputs and resource configurations. Our evaluation investigates whether predictive models can capture these effects from lightweight pre-execution features and whether PRISM can exploit them for stage-local adaptation in a live microservice chain. This leads to the following research questions.

RQ1 – Prediction Foundation. How does vertical CPU scaling affect the execution time and energy consumption of edge microservices across model variants and input resolutions, and can these effects be predicted accurately from lightweight pre-execution features?

RQ2 – Operational Feasibility. What is the per-request decision overhead of PRISM in the deployed microservice chain, and how does cold-start behavior affect latency and energy at pipeline startup?

RQ3 – Pipeline Execution. How well can PRISM satisfy stage-local deadline and offline QoR constraints in a realistic

TABLE I: Offline QoR values measured as mAP for DETECT and word-level accuracy for OCR.

DETECT (MS1)		OCR (MS2)	
Model	QoR	Model	QoR
RT-DETR-x	0.79	PaddleOCRv4	0.61
YOLOv11x	0.75	PaddleOCRv3	0.49
YOLOv5x	0.74	PaddleOCRv2	0.55
YOLOv11n	0.70	PaddleOCRv1	0.41

pipeline, and how does prediction-guided adaptation compare to static configurations in terms of energy and success rate?

We use the ALPR pipeline introduced in Section II as the evaluation workload. This structure reflects a common pattern in edge AI applications where detection is followed by recognition or analysis. Similar multi-stage patterns appear in video analytics, autonomous perception, industrial inspection, and medical imaging, where early-stage decisions can influence downstream cost, latency, and quality.

A. Experimental Setup

Our experiments use a unified measurement workflow that records result quality, per-request execution time, and container-attributed energy for each combination of input, model variant, and CPU allocation. Offline QoR is precomputed per model on labeled data and applied at runtime as a model-level quality constraint in the selection process (Table I). Energy is measured as CPU-package joules using Kepler [19], with RAPL counters attributed to pods and differenced around each request (Section II). Each configuration is repeated multiple times, and we report medians and interquartile ranges (IQR) to reduce sensitivity to noise.

All experiments run on two identical machines with an Intel Core i9-9940X and 64GB RAM under Ubuntu 24.04 LTS. Microservices are containerized and orchestrated using Kubernetes (MicroK8s v1.34.1), and in-place vertical scaling is applied by the kubelet (Section III). Predictive models are trained with scikit-learn [31] and PyMC [32], and per-request decision problems are encoded with a Timefold-based solver [30]. For deployment in the Java-based microservice chain, the trained preprocessing pipeline is exported in ONNX format [33] and loaded at runtime via ONNX Runtime alongside the serialized XGBoost model. This ensures that the preprocessing transformations applied during training are reproduced at inference time. Per-candidate prediction latency remains in the sub-millisecond range on commodity hardware, as reported in RQ2.

Offline QoR is measured as mAP for DETECT and word-level accuracy for OCR on the UC3M-LP dataset [34], which contains 1,975 vehicle images split into 1,580 training and 395 test images. The resulting model-level scores are shown in Table I. To test behavior beyond the profiling distribution in RQ3, we also evaluate on UFPR-ALPR from Laroca et al. [35].

B. RQ1 – Prediction Foundation

Table II reports median and IQR of per-request energy and execution time for the DETECT stage across four model variants, five CPU allocations, and five input resolutions from 3 MP to 64 MP. Execution time decreases substantially with additional CPU allocation, but the speedup depends strongly on input size and model architecture. At 3 MP, the speedup from one to five CPUs ranges from $8\times$ for YOLOv11n to $10\times$ for RT-DETRx, while several 64 MP configurations show diminishing returns beyond three CPUs. Energy follows a less uniform trend. It often decreases with runtime, but the reduction is smaller at high resolutions and can be non-monotonic at intermediate CPU allocations. This confirms that neither execution time nor energy can be explained by CPU allocation alone.

The IQR values show that additional CPU allocation also reduces runtime variability, which is relevant for deadline enforcement. In many configurations, the IQR of execution time shrinks markedly from one to three CPUs. Overall, the measurements show that the best configuration is input- and model-dependent, motivating prediction-guided selection rather than static provisioning.

We next evaluate whether these effects can be predicted from lightweight pre-execution features. All regressors introduced in Section IV are evaluated under identical preprocessing and training conditions. Results are computed on a held-out 10% test split, while the remaining 90% are used for five-fold cross-validation. Table III summarizes the prediction results. XGBoost achieves the lowest relative error across both microservices and targets, with MAPE of 2.6% for time and 4.7% for energy on DETECT, and 6.4% for time and 5.5% for energy on OCR. Linear, Ridge, and Lasso underfit, while polynomial regression and the MLP perform competitively for execution time, but degrade for OCR energy. Bayesian regression provides an uncertainty-aware baseline, but its low point accuracy leads to poor selection performance in RQ3.

Interpretation. Vertical CPU scaling reduces execution time and often also energy, but the effect depends on input resolution and model architecture. These effects are predictable from lightweight features when nonlinear interactions are captured. XGBoost provides the most stable accuracy and is therefore used as the default predictor in PRISM.

C. RQ2 – Operational Feasibility

Figure 5 reports the per-request decision overhead and cold-start behavior of PRISM measured directly in the deployed Java-based microservice chain across 13,528 DETECT and 9,935 OCR requests.

Decision overhead. We measure the overhead of the Timefold-based implementation described in Section IV. Figure 5a reports solving overhead as a percentage of the corresponding inference latency. With XGBoost as the deployed predictor, the median overhead is 30.5% for DETECT and 16.2% for OCR, with 95th percentiles of 59.2% and 42.4%, respectively. This overhead remains below the corresponding inference latency in our deployment, but is large enough

TABLE II: Median and IQR of per-request energy (J) and runtime (s) for the DETECT stage across all detector variants, CPU allocations, and input resolutions (RQ1). IQR values are given in parentheses.

Model	C	3 MP		9 MP		20 MP		32 MP		64 MP	
		$\bar{E}$ (J)	$\bar{T}$ (s)	$\bar{E}$ (J)	$\bar{T}$ (s)	$\bar{E}$ (J)	$\bar{T}$ (s)	$\bar{E}$ (J)	$\bar{T}$ (s)	$\bar{E}$ (J)	$\bar{T}$ (s)
YOLOv1In	1	8.6 (2.0)	0.57 (0.27)	10.1 (1.4)	0.72 (0.12)	14.3 (1.9)	1.04 (0.11)	16.8 (2.2)	1.20 (0.12)	25.4 (3.8)	1.76 (0.13)
	2	6.1 (1.2)	0.14 (0.04)	6.9 (1.5)	0.21 (0.07)	10.2 (1.7)	0.40 (0.05)	13.1 (1.9)	0.60 (0.05)	21.8 (3.7)	1.12 (0.06)
	3	4.5 (0.8)	0.07 (0.02)	5.3 (1.0)	0.14 (0.03)	9.0 (1.5)	0.36 (0.03)	11.7 (1.8)	0.55 (0.03)	20.2 (3.7)	1.06 (0.04)
	4	4.3 (0.7)	0.07 (0.02)	4.9 (0.7)	0.12 (0.02)	8.0 (1.3)	0.31 (0.03)	10.5 (1.5)	0.50 (0.03)	19.2 (3.8)	1.03 (0.04)
	5	4.2 (0.7)	0.07 (0.02)	4.9 (0.7)	0.12 (0.02)	8.0 (1.4)	0.31 (0.03)	10.6 (1.6)	0.50 (0.03)	19.1 (3.8)	1.02 (0.04)
YOLOv1Ix	1	20.5 (2.7)	2.12 (0.21)	19.1 (1.9)	1.84 (0.15)	25.2 (2.8)	2.50 (0.12)	26.9 (3.2)	2.53 (0.12)	36.4 (5.2)	3.24 (0.13)
	2	18.9 (2.9)	1.04 (0.15)	16.5 (1.9)	0.80 (0.07)	23.0 (2.6)	1.21 (0.11)	24.8 (2.8)	1.29 (0.11)	34.7 (5.3)	1.88 (0.12)
	3	16.0 (1.8)	0.54 (0.09)	14.0 (1.7)	0.43 (0.08)	20.3 (2.6)	0.73 (0.08)	21.8 (2.8)	0.84 (0.08)	31.9 (4.8)	1.38 (0.09)
	4	12.4 (1.6)	0.26 (0.06)	10.9 (1.6)	0.24 (0.05)	15.9 (2.1)	0.45 (0.05)	17.9 (2.4)	0.59 (0.06)	28.0 (4.2)	1.11 (0.06)
	5	12.3 (1.9)	0.22 (0.05)	10.7 (1.4)	0.23 (0.03)	16.5 (2.5)	0.45 (0.05)	18.5 (2.8)	0.62 (0.04)	28.6 (5.2)	1.15 (0.06)
YOLOv5x	1	19.7 (2.4)	2.02 (0.11)	18.4 (1.9)	1.80 (0.12)	25.2 (2.8)	2.41 (0.11)	27.0 (3.1)	2.46 (0.11)	36.6 (4.9)	3.12 (0.12)
	2	19.8 (3.2)	1.10 (0.17)	18.4 (1.9)	0.94 (0.10)	25.2 (2.8)	1.36 (0.11)	27.0 (3.0)	1.46 (0.11)	37.3 (4.8)	2.08 (0.11)
	3	15.5 (2.0)	0.49 (0.09)	14.3 (1.8)	0.46 (0.07)	21.1 (2.5)	0.78 (0.09)	23.1 (2.7)	0.92 (0.08)	33.3 (4.8)	1.48 (0.09)
	4	12.7 (1.8)	0.28 (0.05)	12.2 (1.8)	0.30 (0.06)	17.6 (2.3)	0.53 (0.05)	20.0 (2.6)	0.71 (0.06)	29.9 (4.3)	1.25 (0.06)
	5	12.9 (1.9)	0.24 (0.06)	11.7 (1.6)	0.26 (0.03)	17.7 (2.6)	0.49 (0.05)	19.6 (2.9)	0.65 (0.05)	29.5 (4.7)	1.20 (0.06)
RT-DETRx	1	31.4 (2.9)	3.79 (0.41)	33.9 (2.1)	4.04 (0.16)	36.9 (2.5)	4.24 (0.15)	39.5 (2.8)	4.42 (0.14)	48.5 (3.7)	4.94 (0.15)
	2	29.1 (4.5)	1.93 (0.26)	30.0 (4.7)	1.94 (0.27)	33.8 (4.6)	2.09 (0.31)	36.7 (4.9)	2.28 (0.31)	46.6 (8.5)	2.80 (0.31)
	3	24.1 (2.9)	0.98 (0.13)	26.2 (2.7)	1.05 (0.10)	30.5 (3.1)	1.24 (0.11)	33.7 (3.1)	1.42 (0.11)	43.1 (6.2)	1.93 (0.11)
	4	19.8 (2.1)	0.49 (0.08)	21.4 (2.3)	0.55 (0.07)	25.1 (2.7)	0.73 (0.07)	28.0 (3.2)	0.91 (0.07)	38.0 (5.3)	1.42 (0.08)
	5	19.0 (2.3)	0.38 (0.05)	20.9 (2.4)	0.45 (0.05)	24.2 (2.8)	0.62 (0.05)	27.3 (3.3)	0.80 (0.05)	37.2 (5.1)	1.31 (0.07)

to motivate implementation-level optimization. Because the candidate set is bounded by the deployed model variants and CPU options, direct enumeration would be equivalent for this workload and would remove most solver overhead.

With MLP as the predictor, the median overhead drops to 0.6 % for DETECT and 0.4 % for OCR. However, this reduction comes with lower prediction accuracy and weaker selection quality, as shown in RQ1 and RQ3. The results, therefore, indicate a practical trade-off between predictor quality and decision latency in the current Timefold-based implementation. XGBoost yields more accurate predictions but incurs higher solving overhead, while MLP terminates faster at the cost of solution quality. Both variants operate within a bounded overhead determined by the configured termination parameters.

Cold-start behavior. Figures 5b and 5c show the first 25 requests after pod initialization for DETECT and OCR. DETECT exhibits a visible startup effect. The first request takes 8.03 s and consumes 39.7 J, corresponding to $2.3\times$ the warm median duration and $3.0\times$ the warm median energy, before both metrics stabilize by request five. OCR shows negligible cold-start behavior, with the first request at $0.9\times$

the warm median duration and $0.8\times$ the warm median energy. This difference reflects the smaller OCR model footprint and shows that startup behavior is stage-dependent. By preloading candidate model weights at initialization as described in Section II, PRISM avoids model-loading delays during later request processing.

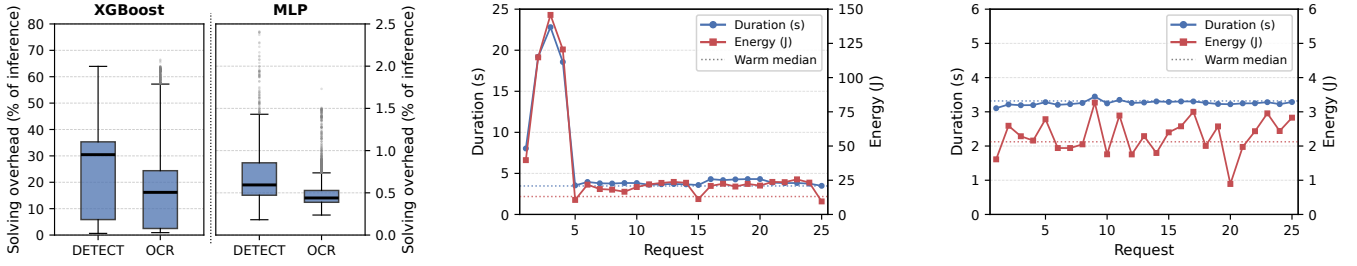
Interpretation. The decision overhead of PRISM is dominated by the Timefold solver rather than predictor inference cost, as the deployed system records service, model-inference, and solving times separately. XGBoost incurs higher overhead than MLP but yields more reliable configuration selections, confirming the trade-off between prediction quality and decision latency identified in RQ1. Cold-start effects are stage-dependent. They are visible for DETECT but negligible for OCR, indicating that preloading is useful for stages with costly model loading but may be unnecessary for lightweight stages. Startup behavior should therefore be measured per stage and separated from later request processing when assessing deadline feasibility.

D. RQ3 – Pipeline Execution

We deploy the full ALPR chain with per-request selection of (m, c) at each stage. Decisions remain stage-local and

TABLE III: Prediction accuracy for per-request execution time (T) and energy (E) across regression candidates for DETECT and OCR. MAPE is the primary decision-relevant metric. Best values within 0.001 of the optimum are bold.

Model	DETECT T					DETECT E					OCR T					OCR E				
	R^2	MAE	MSE	RMSE	MAPE	R^2	MAE	MSE	RMSE	MAPE	R^2	MAE	MSE	RMSE	MAPE	R^2	MAE	MSE	RMSE	MAPE
Bayes	0.676	0.437	0.301	0.548	64.189	0.777	3.355	18.658	4.320	20.497	0.207	0.632	1.123	1.060	98.257	0.668	1.497	3.396	1.843	37.072
Lasso	0.000	0.703	0.930	0.964	107.277	0.599	4.861	33.521	5.790	32.670	0.000	0.754	1.416	1.190	166.105	0.346	1.996	6.700	2.588	53.989
Linear	0.852	0.246	0.138	0.371	36.312	0.958	1.450	3.508	1.873	9.130	0.801	0.321	0.282	0.531	73.946	0.871	0.860	1.321	1.149	22.221
MLP	0.997	0.032	0.003	0.055	4.010	0.981	0.889	1.555	1.247	4.847	0.974	0.100	0.037	0.192	13.499	0.937	0.570	0.642	0.801	11.638
Poly-2	0.997	0.033	0.003	0.056	4.075	0.980	0.927	1.635	1.279	5.220	0.969	0.111	0.044	0.210	18.047	0.931	0.605	0.706	0.841	12.401
Poly-3	0.997	0.027	0.002	0.049	3.087	0.982	0.880	1.528	1.236	4.800	0.972	0.098	0.039	0.198	13.433	0.936	0.580	0.660	0.812	12.195
Ridge	0.852	0.247	0.138	0.371	36.408	0.958	1.450	3.509	1.873	9.130	0.801	0.321	0.282	0.531	73.784	0.871	0.860	1.321	1.149	22.152
XGB	0.998	0.025	0.002	0.047	2.607	0.983	0.870	1.463	1.210	4.730	0.987	0.056	0.018	0.134	6.377	0.984	0.279	0.166	0.408	5.473



(a) Decision overhead as percentage of inference latency for XGBoost and MLP across DETECT and OCR.

(b) Cold-start effect on duration and energy for DETECT over the first 25 requests. Dotted lines indicate the warm median.

(c) Cold-start effect on duration and energy for OCR over the first 25 requests. Dotted lines indicate the warm median.

Fig. 5: Operational feasibility of PRISM measured across 13,528 DETECT and 9,935 OCR requests.

independent across the pipeline. DETECT and OCR models are profiled on UC3M-LP, while the pipeline evaluation uses a mixed Dataset 3 constructed from UC3M-LP and UFPR-ALPR with diverse resolutions and file sizes. Requests are processed with minimum offline QoR thresholds ranging from 60–79 % for DETECT and 40–59 % for OCR. Deadlines are chosen from $\{0.25, 0.5, \dots, 3.25\}$ s. Static baselines execute all fixed (m, c) combinations across the same request set.

All methods are evaluated on 42,000 DETECT and 10,500 OCR requests. Since OCR receives bounding-box crops from DETECT, the effective input size varies per image and detected plate region. Deadlines and offline QoR constraints are applied independently per stage to isolate stage-local adaptation without global budget redistribution. A request is successful if the selected configuration satisfies the stage-local deadline and offline model-level QoR constraint. The request set also includes infeasible combinations of deadlines and QoR thresholds, so even the strongest configuration cannot reach 100 % success. Per-request energy is taken from Kepler’s per-call package-joules and includes failed runs.

Figure 6 compares dynamic PRISM predictors with static configurations. Bars show total energy and energy spent on successful requests, while the line shows the overall success rate. The left part of each plot contains dynamic predictors, and the right part contains fixed model–CPU combinations. This layout makes the central trade-off visible. Static configurations either use little energy but miss many requests due to deadline or offline QoR violations, or achieve high success by using

expensive models and high CPU allocations. Dynamic selection aims to stay near the high-success region while avoiding unnecessary energy use.

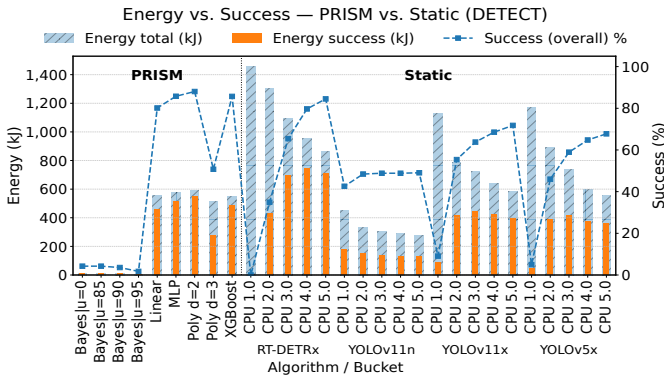
For DETECT, PRISM+XGBoost reaches 85.7 % success at 551.1 kJ with an average allocation of 2.4 CPUs. The strongest static baseline, RT-DETRx with five CPUs, reaches 84.5 % success at 859.9 kJ. This corresponds to a 35.9 % energy reduction while preserving a comparable success rate and using less than half of the average CPU allocation. As shown in Table IV, MLP and Poly $d=2$ reach similar or slightly higher success but consume more energy, while Linear, Poly $d=3$, and Bayesian regression lose substantial success. XGBoost therefore provides the strongest energy-oriented operating point.

For OCR, the static baseline is already close to optimal because PaddleOCRv4 with 1.5 CPUs reaches 90.2 % success at 21.8 kJ. PRISM+XGBoost reaches 91.0 % success at 21.5 kJ with only 0.6 CPUs on average. The gain in energy is therefore small, but the reduction in average CPU allocation is substantial. The OCR plot in Figure 6 also shows why the dynamic gains are smaller than for DETECT. Many fixed OCR configurations already operate in a narrow energy range, while DETECT has larger differences between lightweight and heavy detector configurations.

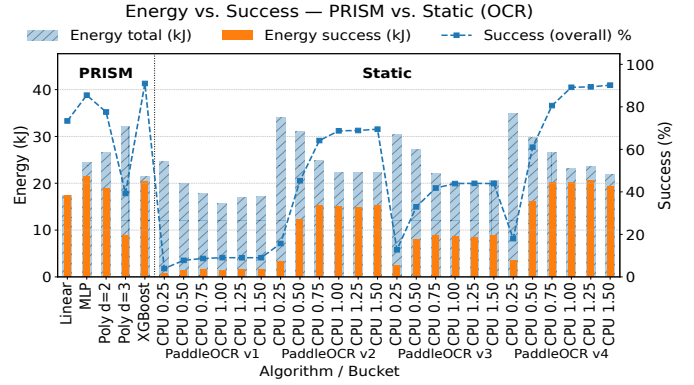
Interpretation. Figure 6 and Table IV show that dynamic selection is most beneficial when the static design space contains both cheap but infeasible and expensive but reliable configurations. This is pronounced for DETECT and weaker

TABLE IV: Summary of PRISM dynamic predictors on Dataset 3 (42,000 DETECT and 10,500 OCR requests). Energy is total per-request energy including failed executions. Reference: RT-DETRx (5 CPU) (DETECT), PaddleOCRv4 (1.5 CPU) (OCR).

Method	DETECT			OCR		
	Succ. (%)	Avg. Cores	Energy (kJ)	Succ. (%)	Avg. Cores	Energy (kJ)
XGBoost	85.7	2.4	551.1	91.0	0.6	21.5
MLP	85.8	2.3	580.0	85.5	0.5	24.4
Poly $d=2$	88.0	2.2	587.3	77.6	0.5	26.5
Poly $d=3$	50.7	1.2	511.4	39.3	0.3	32.1
Linear	80.1	2.7	558.2	73.4	0.4	17.4
<i>Reference: RT-DETRx (5 CPU) (DETECT), PaddleOCRv4 (1.5 CPU) (OCR)</i>						
Static	84.5	5.0	859.9	90.2	1.5	21.8



(a) DETECT stage. PRISM+XGBoost matches the best static success rate while reducing total energy by approximately 36% with lower average CPU demand.



(b) OCR stage. PRISM+XGBoost achieves the best energy–success trade-off among dynamic predictors, with smaller gains than for DETECT because strong static OCR configurations are already close to optimal.

Fig. 6: Static versus dynamic execution on Dataset 3 for both pipeline stages. Prediction-guided, stage-local adaptation reduces energy consumption while maintaining success rates comparable to the strongest static configurations.

for OCR. PRISM+XGBoost matches or slightly exceeds the best static success rates while using substantially lower average CPU allocation in both stages. The DETECT-OCR crop dependency is handled through per-request prediction, allowing PRISM to adapt to actual or estimated crop dimensions instead of relying on a fixed configuration. The results show that stage-local adaptation composes effectively within a multi-stage chain without requiring cross-stage redistribution.

VI. DISCUSSION AND FUTURE WORK

Scope of the Contribution. PRISM is a stage-level adaptation mechanism for edge AI microservices. Its contribution is the integration of container-level energy measurement, prediction-guided model selection, and in-place CPU scaling into a deployable runtime loop. This scope is complementary to chain-level orchestration frameworks such as MARQ. While MARQ selects among alternative service chains and execution paths under application-level constraints, PRISM refines the execution of an individual stage by selecting the model variant and CPU allocation for each request. A chain-level orchestrator can therefore assign per-stage deadlines, quality thresholds, and CPU budgets, while PRISM realizes these bounds locally through predictive in-place adaptation. PRISM does not solve global pipeline scheduling, cross-stage deadline redistribution, or resource allocation across competing pipelines.

Quality and Energy Scope. PRISM uses offline per-model QoR scores as runtime constraints. Configurations are feasible only if the model’s reference quality satisfies the requested threshold. This enables reproducible selection without ground-truth labels at runtime, but it does not guarantee the quality of each individual request. Input difficulty, image content, and distribution shift are therefore not directly captured by the current offline QoR abstraction. Future work should combine offline scores with calibrated online signals such as confidence, temporal consistency, or input-complexity estimates. Similarly,

PRISM optimizes CPU-package energy because CPU allocation is the actuation mechanism studied here. Full-system components such as DRAM, storage, networking, idle base-lines, and model-loading costs remain relevant for full-system accounting, but are not directly controlled by the local CPU-scaling decisions and require platform-specific measurement models.

Runtime Overhead and Model Preloading. The XGBoost-Timefold configuration achieves the best prediction quality but incurs non-negligible overhead, although it remains below the corresponding inference latency in our experiments. Because the candidate set is bounded by the deployed model variants and CPU options, direct enumeration would be equivalent in the evaluated deployment and is the preferred implementation for latency-critical settings. PRISM also assumes that candidate models are preloaded and kept in memory. This avoids model-loading delays during request processing, but increases the persistent memory footprint. Our cold-start results indicate that this trade-off is stage-dependent, as preloading is more useful for stages with costly model initialization than for lightweight stages. Future work should quantify this cost and integrate model loading, eviction, and placement into the orchestration layer.

Generalization and Future Work. The evaluation uses ALPR as a running example because it exposes the core pattern targeted by PRISM. A detection stage produces input-dependent downstream work, alternative model variants provide different quality and cost profiles, and deadline constraints make static overprovisioning inefficient. This pattern appears in many edge AI pipelines, including video analytics, industrial inspection, autonomous perception, and medical imaging. PRISM is therefore not tied to license plate recognition, but to pipelines with measurable pre-execution features, bounded configuration choices, and stage-local quality and deadline constraints. Chain-level orchestration with frame-

works such as MARQ can complement this mechanism by assigning application-level paths and per-stage bounds. To our knowledge, no drop-in adaptive baseline currently combines Kubernetes-based microservice deployment, in-place CPU scaling, per-request model selection, CPU-package energy optimization, and offline QoR constraints. Future work should therefore evaluate this stage-local mechanism in additional domains and study its integration with chain-level orchestration and heterogeneous accelerators.

VII. RELATED WORK

Research on energy- and latency-aware AI inference broadly falls into two areas. The first estimates energy or runtime before execution. The second adapts serving systems at runtime through scaling, scheduling, or approximate model selection.

In the area of predictive methods, Getzner et al. [15] propose layer-wise regression to predict the energy consumption of deep neural networks by composing per-layer costs into architecture-level estimates, thus avoiding full execution on the target device. Their focus is model-level energy prediction rather than runtime integration in containerized microservice pipelines. Inference-serving systems address a related problem. Clipper [36] provides a general-purpose low-latency prediction serving system with caching, batching, and adaptive model selection. INFaaS [37] automates model selection and resource management based on application-level latency and accuracy requirements. Nexus [38] focuses on efficient GPU-cluster serving for DNN applications, while Clockwork [39] exploits predictable DNN inference times to meet tight request-level SLOs. These systems establish important mechanisms for model serving, scheduling, and SLO-aware adaptation. PRISM differs by focusing on containerized edge microservices, CPU-package energy measurements, and in-place vertical CPU scaling under stage-local deadline and offline QoR constraints.

In the area of self-adaptive systems, Razavi et al. [12] study coordinated horizontal and vertical scaling for DL inference pipelines and formulate the end-to-end SLO-driven allocation problem. In a complementary line, Razavi et al. [14] present *Sponge*, which tackles per-request dynamic SLOs by combining in-place vertical scaling, dynamic batching, and request reordering, explicitly accounting for bandwidth-induced variability in mobile networks. Rattihalli et al. [40] explore non-disruptive vertical autoscaling in Kubernetes, while Kaushik et al. [41] study power-aware vertical scaling for deadline-constrained containers. At the model-selection level, Matathammal et al. [42] introduce *EdgeMLBalancer*, an ϵ -greedy strategy that switches between object detectors to balance runtime and accuracy under resource constraints. Nigade et al. [43] guarantee timely serving in dynamic wireless settings with *Jellyfish* by jointly adapting DNNs and inputs to satisfy end-to-end latency SLOs while maintaining accuracy. Reinforcement-learning approaches explore the joint optimization space. Mounesan et al. [44] optimize edge inference parameters, including variant choice and partitioning/offloading decisions, to trade off latency, accuracy, and energy.

Hardware-centric approaches adjust low-level knobs to co-optimize efficiency. EcoEdgeInfer [45] shows that tuning CPU/GPU frequencies and batching can substantially change latency–energy trade-offs on Jetson-class edge devices and uses dynamic configuration to minimize violations under varying conditions. Zhang et al. [46] integrate early exits with DVFS in *E4* to save energy for edge video analytics, adapting exit points and frequencies per frame complexity. Beyond serving stacks, AxIS [47] demonstrates that cross-layer approximations, spanning sensing, networking, storage, and inference, yield significant efficiency gains when quality loss is bounded. For serverless workflows, Rastegar et al. [48] formulate energy-aware scheduling of function chains with fixed deadlines (EneX) via optimization and online scheduling to minimize active energy while avoiding cold starts.

Compared to these lines of work, PRISM targets stage-local adaptation in containerized edge microservice chains. It combines lightweight prediction of execution time and CPU-package energy from pre-execution features with per-request model selection and in-place vertical CPU scaling. This closes the loop from container-level measurement to runtime enforcement. Unlike global serving systems, chain-level schedulers, or vertical scalers for black-box applications, PRISM jointly selects the model variant and CPU allocation of each stage under deadline, energy, and offline QoR constraints. It does not redistribute budgets across stages or optimize cluster-wide resource allocation, but provides a deployable local control mechanism that can complement higher-level orchestrators.

VIII. CONCLUSION

This paper introduced PRISM, a prediction-guided framework that unifies container-level monitoring, regression-based prediction, in-place vertical scaling, and per-stage model selection for microservice chains at the edge. PRISM selects CPU allocations and model variants under energy–time–quality constraints, reducing CPU-package energy for the detection stage by over one-third while maintaining a comparable success rate with less than half the average CPU allocation. In an evaluation with more than 52,000 requests across both stages of an ALPR pipeline, PRISM adapts models and CPU allocations per request, outperforming strong static baselines for detection and matching near-static-best performance for recognition with lower average CPU allocation. The approach is practical, directly deployable on Kubernetes, and grounded in container-level measurements, making it reproducible for edge microservice deployments. PRISM shows that combining accurate prediction with in-place scaling and per-stage model selection is an effective mechanism for reliable, energy-efficient execution of latency-sensitive AI services at the edge.

IX. ACKNOWLEDGMENT

This work has been funded by the European Commission Horizon Europe Smart Networks and Services Joint Undertaking (SNS JU) EXIGENCE Project (Grant Agreement No. 101139120).

REFERENCES

- [1] M. Satyanarayanan, "The emergence of edge computing," *Computer*, vol. 50, no. 1, pp. 30–39, 2017. DOI: 10.1109/MC.2017.9
- [2] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge Computing: Vision and Challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016. DOI: 10.1109/JIOT.2016.2579198
- [3] T. Wang, J. Guo, B. Zhang, G. Yang, and D. Li, "Deploying AI on Edge: Advancement and Challenges in Edge Intelligence," *Mathematics*, vol. 13, no. 11, p. 1878, 2025. DOI: 10.3390/math13111878
- [4] W. Liu, F. Lombardi, and M. Shulte, "A Retrospective and Prospective View of Approximate Computing," *Proceedings of the IEEE*, vol. 108, no. 3, pp. 394–399, 2020. DOI: 10.1109/JPROC.2020.2975695
- [5] H. B. Barua and K. C. Mondal, "Approximate Computing: A Survey of Recent Trends—Bringing Greenness to Computing and Communication," *Journal of The Institution of Engineers (India): Series B*, vol. 100, no. 6, pp. 619–626, 2019. DOI: 10.1007/s40031-019-00418-8
- [6] V. K. Chippa, S. T. Chakradhar, K. Roy, and A. Raghunathan, "Analysis and characterization of inherent application resilience for approximate computing," in *Proceedings of the 50th Annual Design Automation Conference*, ser. DAC, ACM, 2013, pp. 1–9. DOI: 10.1145/2463209.2488873
- [7] U. Gropengießer, T. Reuter, D. Schön, O. Abboud, X. Xiao, and M. Mühlhäuser, "Towards Predictable Energy and Runtime Estimation for Vertically Scaled Edge AI Inference," in *International Conference on Pervasive Computing and Communications Workshops and other Affiliated Events (PerCom Workshops)*, IEEE, 2026, pp. 1–6. DOI: 10.1109/PerComWorkshops68308.2026.11585449
- [8] P. Padala, K. G. Shin, X. Zhu, M. Uysal, Z. Wang, S. Singhal, A. Merchant, and K. Salem, "Adaptive control of virtualized resources in utility computing environments," *SIGOPS Oper. Syst. Rev.*, vol. 41, no. 3, pp. 289–302, 2007. DOI: 10.1145/1272998.1273026
- [9] J. Krzywda, A. Ali-Eldin, T. E. Carlson, P.-O. Östberg, and E. Elmroth, "Power-performance tradeoffs in data center servers: DVFS, CPU pinning, horizontal, and vertical scaling," *Future Generation Computer Systems*, vol. 81, pp. 114–128, 2018. DOI: 10.1016/j.future.2017.10.044
- [10] P. Pandey and D. Pompili, "MobiDiC: Exploiting the Untapped Potential of Mobile Distributed Computing via Approximation," in *International Conference on Pervasive Computing and Communications*, ser. PerCom, vol. 38, IEEE, 2016, pp. 1–9. DOI: 10.1109/PERCOM.2016.7456515
- [11] Y. Li, Y. Chen, T. Lan, and G. Venkataramani, "MobiQoR: Pushing the Envelope of Mobile Edge Computing Via Quality-of-Result Optimization," in *International Conference on Distributed Computing Systems*, ser. ICDCS, IEEE, 2017, pp. 1261–1270. DOI: 10.1109/ICDCS.2017.54
- [12] K. Razavi, M. Salmani, M. Mühlhäuser, B. Koldehofe, and L. Wang, *A Tale of Two Scales: Reconciling Horizontal and Vertical Scaling for Inference Serving Systems*, 2024. DOI: 10.48550/arXiv.2407.14843
- [13] R. S. Kannan, L. Subramanian, A. Raju, J. Ahn, J. Mars, and L. Tang, "GrandSLAM: Guaranteeing SLAs for jobs in microservices execution frameworks," in *Proceedings of the 14th EuroSys Conference*, ser. EuroSys, Dresden, Germany: ACM, 2019, pp. 1–16. DOI: 10.1145/3302424.3303958
- [14] K. Razavi, S. Ghafouri, M. Mühlhäuser, P. Jamshidi, and L. Wang, "Sponge: Inference Serving with Dynamic SLOs Using In-Place Vertical Scaling," in *Proceedings of the 4th Workshop on Machine Learning and Systems*, ser. EuroMLSys, ACM, 2024, pp. 184–191. DOI: 10.1145/3642970.3655833
- [15] J. Getzner, B. Charpentier, and S. Günemann, *Accuracy is not the only Metric that matters: Estimating the Energy Consumption of Deep Learning Models*, 2023. DOI: 10.48550/arXiv.2304.00897
- [16] P. Kurp, "Green computing," *Commun. ACM*, vol. 51, no. 10, pp. 11–13, 2008. DOI: 10.1145/1400181.1400186
- [17] C. Centofanti, J. Santos, V. Gudepu, and K. Kondepu, "Impact of power consumption in containerized clouds: A comprehensive analysis of open-source power measurement tools," *Computer Networks*, vol. 245, 2024. DOI: 10.1016/j.comnet.2024.110371
- [18] U. Gropengießer, E. Dietz, F. Brandherm, A. Doula, O. Abboud, X. Xiao, and M. Mühlhäuser, "MARQ: Engineering Mission-Critical AI-Based Software with Automated Result Quality Adaptation," in *International Conference on Software Engineering*, ser. ICSE, IEEE/ACM, 2025, pp. 1934–1946. DOI: 10.1109/ICSE55347.2025.00182
- [19] M. Amaral, H. Chen, T. Chiba, R. Nakazawa, S. Choochotkaew, E. K. Lee, and T. Eilam, "Kepler: A Framework to Calculate the Energy Consumption of Containerized Applications," in *16th International Conference on Cloud Computing*, ser. CLOUD, IEEE, 2023, pp. 69–71. DOI: 10.1109/CLOUD60044.2023.00017
- [20] K. N. Khan, M. Hirki, T. Niemi, J. K. Nurminen, and Z. Ou, "RAPL in Action: Experiences in Using RAPL for Power Measurements," *ACM Trans. Model. Perform. Eval. Comput. Syst.*, vol. 3, no. 2, pp. 1–26, 2018. DOI: 10.1145/3177754
- [21] R. Padilla, S. L. Netto, and E. A. B. Da Silva, "A Survey on Performance Metrics for Object-Detection Algorithms," in *International Conference on Systems, Signals and Image Processing*, ser. IWSSIP, IEEE, 2020, pp. 237–242. DOI: 10.1109/IWSSIP48289.2020.9145130
- [22] J. Baek, G. Kim, J. Lee, S. Park, D. Han, S. Yun, S. J. Oh, and H. Lee, "What Is Wrong With Scene Text Recognition Model Comparisons? Dataset and Model Analysis," *IEEE/CVF International Conference on Computer Vision*, ICCV, pp. 4714–4722, 2019. DOI: 10.1109/ICCV.2019.00481
- [23] I. Goiri, R. Bianchini, S. Nagarakatte, and T. D. Nguyen, "ApproxHadoop: Bringing Approximations to MapReduce Frameworks," in *International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. 4, vol. 50, ACM, 2015, pp. 383–397. DOI: 10.1145/2775054.2694351
- [24] Z. Wen, D. L. Quoc, P. Bhatotia, R. Chen, and M. Lee, "ApproxIoT: Approximate Analytics for Edge Computing," in *International Conference on Distributed Computing Systems*, ser. ICDCS, IEEE, 2018, pp. 411–421. DOI: 10.1109/ICDCS.2018.00048
- [25] G. Hu, S. Rigo, D. Zhang, and T. Nguyen, "Approximation with Error Bounds in Spark," in *International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems*, ser. MASCOTS, IEEE, 2019, pp. 61–73. DOI: 10.1109/MASCOTS.2019.00017
- [26] S. Luo, H. Xu, C. Lu, K. Ye, G. Xu, L. Zhang, Y. Ding, J. He, and C. Xu, "Characterizing microservice dependency and performance: Alibaba trace analysis," in *Proceedings of the 2021 ACM Symposium on Cloud Computing*, ser. SoCC '21, Seattle, WA, USA: ACM, 2021, pp. 412–426. DOI: 10.1145/3472883.3487003
- [27] D. Beyer and P. Wendler, "CPU Energy Meter: A Tool for Energy-Aware Algorithms Engineering," in *Tools and Algorithms for the Construction and Analysis of Systems: 26th International Conference, TACAS 2020, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2020, Proceedings, Part II*, Berlin, Heidelberg: Springer-Verlag, 2020, pp. 126–133. DOI: 10.1007/978-3-030-45237-7_8
- [28] S. Choochotkaew, C. Wang, H. Chen, T. Chiba, M. Amaral, E. K. Lee, and T. Eilam, *A Robust Power Model Training Framework for Cloud Native Runtime Energy Metric Exporter*, 2024. DOI: 10.48550/arXiv.2407.00878
- [29] Kubernetes, *Resize CPU and Memory Resources assigned to Containers*. Accessed: Aug. 21, 2026. [Online]. Available: <https://kubernetes.io/docs/tasks/configure-pod-container/resize-container-resources>
- [30] G. De Smet, L. Petrovický, and o. s. contributors open source, *Timefold documentation*, 2023. Accessed: May 28, 2026. [Online]. Available: <https://timefold.ai>
- [31] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, "Scikit-learn: Machine Learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011, ISSN: 1532-4435.
- [32] J. Salvatier, T. V. Wiecki, and C. Fonnesbeck, "Probabilistic programming in Python using PyMC3," *PeerJ Computer Science*, vol. 2, e55, 2016. DOI: 10.7717/peerj-cs.55
- [33] O. r. developers, *ONNX Runtime*, 2021. Accessed: May 5, 2026. [Online]. Available: <https://onnxruntime.ai/>
- [34] Á. Ramajo-Ballester, J. M. Armingol Moreno, and A. de la Escalera Hueso, "Dual license plate recognition and visual features encoding for vehicle identification," *Robotics and Autonomous Systems*, vol. 172, p. 104608, 2024. DOI: 10.1016/j.robot.2023.104608
- [35] R. Laroca, E. Severo, L. A. Zanlorensi, L. S. Oliveira, G. R. Gonçalves, W. R. Schwartz, and D. Menotti, "A Robust Real-Time Automatic License Plate Recognition Based on the YOLO Detector," in *International Joint Conference on Neural Networks*, ser. IJCNN, 2018, pp. 1–10. DOI: 10.1109/IJCNN.2018.8489629

- [36] D. Crankshaw, X. Wang, G. Zhou, M. J. Franklin, J. E. Gonzalez, and I. Stoica, "Clipper: A Low-Latency Online Prediction Serving System," in *14th USENIX Symposium on Networked Systems Design and Implementation*, ser. NSDI, 2017, pp. 613–627, ISBN: 978-1-931971-37-9.
- [37] F. Romero, Q. Li, N. J. Yadwadkar, and C. Kozyrakis, "INFaaS: Automated Model-less Inference Serving," in *USENIX Annual Technical Conference (USENIX ATC 21)*, 2021, pp. 397–411, ISBN: 978-1-939133-23-6.
- [38] H. Shen, View Profile, L. Chen, View Profile, Y. Jin, View Profile, L. Zhao, View Profile, B. Kong, View Profile, M. Philipose, View Profile, A. Krishnamurthy, View Profile, R. Sundaram, and View Profile, "Nexus: A GPU cluster engine for accelerating DNN-based video analysis," in *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, ser. SOSP, ACM, 2019, pp. 322–337. DOI: 10.1145/3341301.3359658
- [39] A. Gujarati, R. Karimi, S. Alzayat, W. Hao, A. Kaufmann, Y. Vigfusson, and J. Mace, "Serving DNNs like Clockwork: Performance Predictability from the Bottom Up," in *14th USENIX Symposium on Operating Systems Design and Implementation*, ser. OSDI, 2020, pp. 443–462, ISBN: 978-1-939133-19-9.
- [40] G. Rattihalli, M. Govindaraju, H. Lu, and D. Tiwari, "Exploring Potential for Non-Disruptive Vertical Auto Scaling and Resource Estimation in Kubernetes," in *12th International Conference on Cloud Computing*, ser. CLOUD, 2019, pp. 33–40. DOI: 10.1109/CLOUD.2019.00018
- [41] P. Kaushik, S. Raghavendra, and M. Govindaraju, "A Study of Contributing Factors to Power Aware Vertical Scaling of Deadline Constrained Applications," in *2022 IEEE 15th International Conference on Cloud Computing*, ser. CLOUD, IEEE, 2022, pp. 500–510. DOI: 10.1109/CLOUD55607.2022.00073
- [42] A. Matathammal, K. Gupta, L. Lavanya, A. V. Halgatti, P. Gupta, and K. Vaidyanathan, *EdgeMLBalancer: A Self-Adaptive Approach for Dynamic Model Switching on Resource-Constrained Edge Devices*, 2025. DOI: 10.48550/arXiv.2502.06493
- [43] V. Nigade, P. Bauszat, H. Bal, and L. Wang, "Jellyfish: Timely Inference Serving for Dynamic Edge Networks," in *Real-Time Systems Symposium*, ser. RTSS, IEEE, 2022, pp. 277–290. DOI: 10.1109/RTSS55097.2022.00032
- [44] M. Mounesan, X. Zhang, and S. Debroy, *Infer-EDGE: Dynamic DNN Inference Optimization in 'Just-in-time' Edge-AI Implementations*, arXiv:2501.18842 [cs], 2025. DOI: 10.48550/arXiv.2501.18842
- [45] S. P. Rachuri, N. Shaik, M. Choksi, and A. Gandhi, "EcoEdgeInfer: Dynamically Optimizing Latency and Sustainability for Inference on Edge Devices," in *Symposium on Edge Computing*, ser. SEC, IEEE/ACM, 2024, pp. 191–205. DOI: 10.1109/SEC62691.2024.00023
- [46] Z. Zhang, Y. Zhao, M.-C. Chang, C. Lin, and J. Liu, *E4: Energy-Efficient DNN Inference for Edge Video Analytics Via Early-Exit and DVFS*, 2025. DOI: 10.48550/arXiv.2503.04865
- [47] S. K. Ghosh, A. Raha, and V. Raghunathan, "Approximate inference systems (AxIS): End-to-end approximations for energy-efficient inference at the edge," in *Proceedings of the International Symposium on Low Power Electronics and Design*, ser. ISLPED '20, New York, NY, USA: ACM/IEEE, 2020, pp. 7–12. DOI: 10.1145/3370748.3406575
- [48] S. H. Rastegar, H. Shafiei, and A. Khonsari, "EneX: An Energy-Aware Execution Scheduler for Serverless Computing," *IEEE Transactions on Industrial Informatics*, vol. 20, no. 2, pp. 2342–2353, 2024. DOI: 10.1109/TII.2023.3290985